

Sprint 4 – Individual Retrospective Kamogelo Mosehle

Software Design

What I Built

PayFast fixes. I resolved two compounding issues on the deployed site. A 403 error was caused by `NODE_ENV=production` setting `IS_SANDBOX = false`, sending sandbox credentials to the live PayFast URL. I decoupled sandbox mode from `NODE_ENV` using a dedicated `PAYFAST_SANDBOX` server env var and `VITE_PAYFAST_SANDBOX` on the client. The payment success page was also hanging because `import.meta.env.DEV` is always `false` in a Vite production build, so polling ran instead of `confirm-dev`. Replacing it with `VITE_PAYFAST_SANDBOX` fixed this for both environments.

Redesigned booking flow. The buyer previously booked a collection slot immediately after payment. I redesigned this so the seller receives an in-app notification when a buyer pays, books drop-off first, then the buyer receives a notification to book collection. The backend enforces this ordering, a collection booking is rejected if no drop-off exists yet. Slot filtering was also fixed: sellers see slots at least one hour from now, buyers see slots from the seller's drop-off time onwards.

Notification system. I created the `notifications` table with RLS policies and added it to Supabase Realtime alongside `messages` and `facility_bookings`. Building on existing message and booking notification infrastructure, I added categorised types (payment, booking, and message), visually distinguished in the notifications panel. The navbar bell icon now shows a live unread count badge updated via a Realtime subscription.

Database and testing. I set up `notifications` table RLS policies, set up a `generate_weekly_slots` PostgreSQL function via `pg_cron` to auto-generate slots every Sunday, and fixed a failing `PaymentPage` Jest test by setting `process.env.VITE_PAYFAST_SANDBOX = "true"`.

What I Found Difficult

The booking flow redesign required the buyer's slot UI to fetch the seller's drop-off time first and pass it as a query parameter, getting this dependency chain right without race conditions took several iterations. The PayFast debugging was also slow because three bugs were compounding across the different environments, each fix revealing the next.

What I Would Do Differently

I would define the booking sequence and get the ordering constraint written down upfront, which would have prevented the original incorrect flow.

How My Work Fits the Overall System

The enforced drop-off before collection ordering gives the campus facility staff operational reliability, every collection slot now has a confirmed drop-off behind it. Combined with the live notification system, both parties are guided through each transaction step automatically.